

Contents

1	知识地图·大语言模型全景（M0–M9）	3
1.1	判断”真的懂了”的三条硬标准（本学科通用）	3
1.2	M0·数学与机器学习地基	3
1.3	M1·语言模型史与前 Transformer 时代	4
1.4	M2·分词与表示	4
1.5	M3·Transformer 架构与内部机制	5
1.6	M4·训练：数据、优化、系统	5
1.7	M5·推理与部署	6
1.8	M6·后训练：从”会说话”到”会做事”	6
1.9	M7·评测：把”感觉变好了”变成”确实变好了”	7
1.10	M8·智能体：工具、记忆、harness、长程任务	7
1.11	M9·前沿与安全（持续跟进区，不是一次性学完）	8
1.12	模块 ↔ 课题 ↔ 周次 对照	8
1.13	与课程（CORE100299）六板块的映射	9
2	对标教学资源清单（2026-09-17 核实）	10
2.1	一、主线三件套（工程 + 判分，本学科骨架）	10
2.2	二、理论线的”对标名师”（补”讲”的深度）	11
2.3	三、评测与环境类（M7/M8 的判分件）	16
2.4	四、明确标注的坑（用前必读）	18
2.5	五、三师制使用方法（每个知识点的标准动作）	18
3	知行合一规程（本学科教学法核心）	20
3.1	一、把”知行合一”变成可执行的四件咬合	20
3.2	二、知识卡规范（每个知识点一张，落 知识卡/ 或课题目录）	20
3.3	三、讲课节拍（每周两个时段，各 1.5–3h）	21
3.4	四、代码级提问（“知”与”行”的咬合点，本学科独创）	21
3.5	五、“真的懂了”的三条硬标准（与《知识地图》一致，此处为验收动作）	22
3.6	六、反讲机制（费曼 + 红队）	22

3.7 七、与其它章程的关系	22
4 实训路线图 · 16 周 × 11 课题 × 唐杰 7 条 × CS336 映射	24
4.1 一、目标（一句话）	24
4.2 二、唐杰 7 条 ↔ 本学科课题 ↔ 判分来源（主映射表）	24
4.3 三、16 周课表（周次 / 课题 / 交付物 / GPU 需求）	25
4.4 四、11 课题总表（手写要点 / 判分来源 / 通过证据）	29
4.5 五、规模三档（零预算版）	31
4.6 六、跨学科搭桥（讲课时主动用）	32
5 造轮子边界 · 谁写代码（本学科最高契约）	33
5.1 一、三条铁律	33
5.2 二、手写清单（学习目标，共 10 项）	33
5.3 三、代写清单（脚手架，Agent 负责，共 8 项）	34
5.4 四、协作区（一起定，你拍板）	35
5.5 五、违规示例（判例，会长大）	35
5.6 六、边界自检（每次动手前问一遍）	36
6 AI 使用红线（本学科版）	37
6.1 一、最高原则	37
6.2 二、手写区：Agent 应当做（SHOULD）	37
6.3 三、手写区：Agent 禁止做（SHOULD NOT，照抄 CS336 并加严）	37
6.4 四、脚手架区：Agent 应当做（且必须做完）	38
6.5 五、例外与补偿（本学科特有，CS336 没有）	38
6.6 六、教学法提示（照抄 CS336 的方法论，本学科已内化）	38
6.7 七、学术诚信与“知行合一”的关系（本学科的立场）	38
6.8 八、越界举报（自检清单，每轮自问）	39
7 算力与成本预算（零预算版）	40
7.1 一、免费资源池（2026-09-17 核实）	40
7.2 二、三档规模（零预算下的能力边界）	41
7.3 三、成本估算（标记为估算，必须在课题04 用实测替换）	41
7.4 四、每周配额分配（16 周）	42
7.5 五、硬纪律（出过事故的领域）	42
7.6 六、可选升级路径（仅在零预算实在卡住时启动，需用户拍板）	43
8 判分与验收标准	44
8.1 一、三层判分（每一层都必须留下原始输出）	44
8.2 二、通用红线（继承总纲 + 本学科加严）	44
8.3 三、每课题的“通过证据”清单（缺一不可）	45

8.4	四、成绩口径（对齐唐杰作业的 40% / 60%）	46
8.5	五、自欺自查清单（结项前必过）	47
9	上游锁定清单（本学科只读依赖）	48
9.1	一、骨架依赖（必须锁，判分与训练都靠它）	48
9.2	二、知识线依赖（只读，不入仓）	49
9.3	三、许可与合规注意	51
9.4	四、重审机制	52

1 知识地图 · 大语言模型全景 (M0–M9)

立图 2026-09-17。本文件回答一个问题：要把大语言模型”彻底搞懂”，到底要懂哪些东西。每个模块都是”四个咬合件”：核心问题 → 对标材料 → 亲手实验 → 判分证据。只讲不做 = 没学（见《知行合一规程.md》）；只做不讲 = 学歪。

用法：模块不是线性章节，但顺序有依赖（M0→M1→M2→M3→M4→M5→M6→M7→M8→M9）。每周讲哪个模块，见《实训路线图.md》的”知识讲解锚点”列。

1.1 判断”真的懂了”的三条硬标准（本学科通用）

1. 能讲清：不用术语、用大白话讲给外行听，且能回答”为什么不是别的做法”。
2. 能指出代码位置：说出这个概念在本仓 手写/ 或上游 nanochat/CS336 里具体是哪个函数、哪几行。
3. 能改坏并修好：故意改坏（去掉 mask、换掉归一化、打乱位置编码）→ 预测现象 → 跑实验验证 → 修回。（第 3 条是”知”与”行”的分界线；只会背定义的人在第三步一定翻车。）

1.2 M0 · 数学与机器学习地基

- 核心问题：梯度到底怎么传过 100 层？为什么归一化能让训练稳下来？交叉熵、困惑度、bpb 三者的关系是什么？参数量/FLOPs/显存怎么估？
- 关键概念：链式法则与计算图、雅可比、损失函数（MSE/交叉熵）、梯度下降族、正则化、数值稳定性、信息量与熵、复杂度记账（6ND 公式）
- 对标：李沐《动手学深度学习》ch2–ch5、ch11（优化算法）；Karpathy micrograd 视频；概率论课题01–08（已建成，直接搭桥）

- 亲手实验：手写 micrograd（标量自动微分，200 行内）→ 手写 MLP 在 MNIST 子集上训到 >95%
- 判分证据：与 PyTorch autograd 的数值梯度对照表（误差 < 1e-6）+ 参数量/FLOPs 估算脚本实测偏差
- 常见误解：把 loss 当“正确率”；以为“梯度消失”是 bug 而不是结构性质；以为 d2l 的 MXNet 代码不用改

1.3 M1 · 语言模型史与前 Transformer 时代

- 核心问题：Transformer 解决了什么前人的痛点？为什么 attention 取代了 RNN？“语言模型”这个词 20 年里的含义变化？
- 关键概念：n-gram 与平滑、词袋、word2vec/GloVe、RNN/LSTM/GRU、seq2seq + 注意力、Bahdanau/Luong attention、并行化的动机
- 对标：李沐 d2l ch8–ch10（RNN → 注意力）；CS336 lecture 01（概览与分词）；《Attention Is All You Need》原文（入论文精读）
- 亲手实验：手写 bigram/trigram 语言模型算困惑度 → 手写一个最小 RNN LM → 对比同参数量的单层 Transformer
- 判分证据：三者在同一语料上的 bpb 对照表 + “RNN 为什么慢”的显式计时（逐步 vs 并行）
- 常见误解：以为“注意力=Transformer”；以为 attention 是 2017 才发明的（其实 2014 就有）

1.4 M2 · 分词与表示

- 核心问题：为什么不用字符？为什么不用整词？词表大小怎么权衡？中文/代码/数学符号怎么处理？
- 关键概念：BPE（训练循环、merge 规则、特殊 token、bytes 回退）、WordPiece、Unigram、词表大小 ↔ 序列长度 ↔ 参数量的三方权衡、压缩率
- 对标：CS336 A1 tokenizer 部分；nanochat tokenizer.py；HF LLM Course ch2（“Implement BPE/WordPiece/Unigram from the ground up”）
- 亲手实验：手写 BPE（训练 + 编码 + 流式编解码往返）→ 在 TinyStories 与中文语料上各训一个词表 → 测压缩率
- 判分证据：tests/test_tokenizer.py、test_train_bpe.py 全绿 + 压缩率表（中英文分别）+ 往返一致性
- 常见误解：以为 token 就是“词”；忽略中文一个汉字可能被切成 2–3 个 byte token 导致实际序列变长

1.5 M3 · Transformer 架构与内部机制

- 核心问题: attention 到底在算什么? 为什么需要多头? 位置编码为什么从正弦换成 RoPE? RMSNorm 凭什么比 LayerNorm 更常用? 残差在防什么?
- 关键概念: QKV 与缩放点积、因果 mask、多头与 GQA/MQA、RoPE (含外推)、RMSNorm、pre-norm vs post-norm、FFN/SwiGLU、权重共享、初始化与 QK-norm、参数量分解
- 对标: CS336 lecture 03-04 + A1 model 部分; rasbt 《LLMs-from-scratch》ch3-ch4 (含 MoE/GQA/MLA/SWA 附录); 李宏毅 GenAI-ML; nanochat gpt.py
- 亲手实验: 手写完整 Transformer 层 (含 RoPE/RMSNorm/SwiGLU) → 形状测试 → 改坏实验: 去掉因果 mask、把 RoPE 换成可学习位置嵌入、去掉残差, 各跑一次看现象
- 判分证据: tests/test_model.py、test_nn_utils.py 全绿 + 三组“改坏”实验的 loss 曲线与现象记录 (进 证据/)
- 常见误解: 以为 attention 是“可解释的检索”; 以为参数越多越好; 忽略 pre-norm 对稳定性的决定性作用

1.6 M4 · 训练: 数据、优化、系统

- 核心问题: 为什么同样的模型有人训得动有人训崩? Scaling law 怎么指导算力分配? 多卡为什么不是线性加速?
- 关键概念: 预训练目标与数据配比、去重与过滤、AdamW (解耦权重衰减)、Muon、LR schedule (warmup+cosine)、梯度裁剪、混合精度 (bf16/fp16/fp32 与 GradScaler)、梯度累积、DDP/FSDP/TP/PP、通信开销与扩展效率、Chinchilla 与 compute-optimal、`--target_flops`
- 对标: CS336 A2/A3 + lecture 05-11; nanochat optim.py/base_train.py /runs/scaling_laws.sh; HF Ultra-Scale Playbook (分布式, 待核); 李沐 d2l ch11-ch12
- 亲手实验: 手写 AdamW 与 LR 调度 → 单卡固定算力下的 depth sweep (d4/d6/d8/d10) → 拟合幂律外推 d12 → 用真实 d12 训练验证预测 (打脸链路) → 双卡 DDP 测扩展效率
- 判分证据: tests/test_optimizer.py 全绿; sweep 原始数据 + 拟合图 + 外推预测存档 (预测先写死再跑); 扩展效率曲线 + 通信占比
- 常见误解: 以为“loss 不降就是数据问题”; 把显存不足当成硬件不够 (其实是批大小/精度/累积没调); 把 DDP 当免费加速

1.7 M5 · 推理与部署

- 核心问题：为什么生成是逐个 token 的？KV cache 省了什么、代价是什么？量化会掉多少点？吞吐与延迟怎么权衡？
- 关键概念：自回归解码、KV cache、批处理与 continuous batching、PagedAttention、采样策略（temperature/top-p/top-k）、量化（int8/int4/GPTQ/AWQ）、投机解码、显存占用分解、token 成本
- 对标：nanochat engine.py（KV cache 实现）+ infer_bench.py；CS336 lecture 10（Inference）；rasbt ch2（生成与 KV caching）
- 亲手实验：从零写 KV cache 的自回归生成 → 对比有无 cache 的耗时/显存 → 手写一个贪心 vs top-p 采样对照 → 测不同 batch size 的吞吐
- 判分证据：tests/test_engine.py 全绿 + 延迟/吞吐/显存三张图 + “cache 换来多少倍加速”的实测数字
- 常见误解：以为推理慢是因为“模型太大”（常是没批处理/没 cache）；以为量化无损

1.8 M6 · 后训练：从“会说话”到“会做事”

- 核心问题：SFT、DPO、RLVR 各自在补什么能力？为什么 RLVR 能提升数学/代码却可能伤害通用能力？reward hacking 长什么样？
- 关键概念：指令微调与 chat template、损失掩码（只算 answer 部分）、奖励模型、PPO、DPO 的隐式奖励、GRPO 的组内归一化优势、可验证奖励（verifiable reward）、拒绝采样、蒸馏、合成数据、KL 正则与过度优化
- 对标：CS336 A5（含 test_grpo.py、GSPO/MaxRL）；rasbt 《Reasoning from Scratch》ch3–ch8（先建评测→推理时扩展→self-refinement→GRPO→改进 GRPO→蒸馏）；Nathan Lambert 《RLHF Book》（免费在线，17 章）；HF LLM Course ch11–12；minimind train_dpo/train_grpo.py；nano-aha-moment（单文件单卡）
- 亲手实验：同一基座上做 SFT / DPO / GRPO 三变体 → 同一评测口径下对照 → 手写 GRPO 损失并逐项注释（advantage 从哪来、为什么要除标准差）→ 制造一次 reward hacking 并记录现象
- 判分证据：tests/test_grpo.py 全绿 + 三方对照表（同基座/同评测/同种子策略）+ reward 曲线与失败样本分析
- 常见误解：以为 DPO 不用 reward model 就“更简单”；以为 RL 一定比 SFT 强；把训练 reward 上升当成能力提升（不做 held-out 评测就是自欺）

1.9 M7 · 评测：把”感觉变好了”变成”确实变好了”

- 核心问题：benchmark 分数可信吗？污染怎么判断？为什么必须固定口径？评测方差有多大？
- 关键概念：bpb / perplexity / CORE / 下游基准（GSM8K/MATH-500/MMLU/HumanEval）、pass@k、评测污染、few-shot 与 prompt 敏感性、方差与置信区间、LLM-as-judge 的偏置、可验证奖励 vs 学习型奖励
- 对标：rasbt 《Reasoning from Scratch》ch3（先建评测再上技术，全书教学法基石）；nanochat core_eval.py（单文件 CORE）；CS336 lecture 12；Berkeley Agentic AI MOOC “Agent Evaluation”；lm-evaluation-harness
- 亲手实验：写一个”数学题判定器”（解析 + 数值比对 + 容错）→ 测它在 200 道题上的准确率与假阳性 → 给自己的两个模型跑同一套评测并给误差棒
- 判分证据：判定器的单测 + 假阳性/假阴性样例分析 + 带误差棒的对照表（不许只报均值）
- 常见误解：只看一个 benchmark 下结论；忽略 prompt 格式带来的巨大差异；把测试集当验证集反复调

1.10 M8 · 智能体：工具、记忆、harness、长程任务

- 核心问题：LLM 从”回答问题”到”完成任务”缺什么？harness 为什么比模型本身更能决定成败？self-judge 什么时候有效、什么时候是自欺？
- 关键概念：工具调用/函数调用、ReAct 循环、context engineering（Karpathy/李宏毅都专讲）、记忆与压缩、harness 设计（沙箱、权限、反馈信号）、长程任务的误差累积、多智能体、self-judge / self-refine 的边界条件、可验证环境
- 对标：李宏毅 GenAI-ML 《Context Engineering》专讲；Berkeley LLM Agents MOOC + Agentic AI MOOC（含 “Post-Training Verifiable Agents”）；HF Agents Course（三大框架 + 排行榜）；rasbt 《Reasoning from Scratch》ch5（self-refinement）；PrimeIntellect-ai/verifiers、terminal-bench、SWE-bench
- 亲手实验：手写最小 ReAct 循环（不用框架）→ 给它 2 个工具（计算器 + 代码执行）→ 记录失败轨迹分类 → 加 self-judge 再跑，做消融
- 判分证据：轨迹日志（成功/失败分类）+ self-judge 有无的成功率差 + “什么时候 self-judge 有害”的案例
- 常见误解：以为 Agent 的瓶颈在模型智力（常在你的工具接口与反馈信号）；以为 self-judge 免费提升（它会放大原有错误）

1.11 M9 · 前沿与安全（持续跟进区，不是一次性学完）

- 核心问题：MoE/长上下文/多模态改变了什么？对齐与安全是“加一层”还是“改训练目标”？持续学习与“AI 训 AI”可行到什么程度？
- 关键概念：MoE 稀疏激活、长上下文外推与记忆、多模态对齐、可解释性、红队与越狱、幻觉的成因、评测式安全保障、合成数据闭环、self-play、灾难性遗忘、模型合并
- 对标：CS336 lecture 04 (MoE) + 论文精读清单；Berkeley Agentic AI MOOC（安全与红队）；opendilab/awesome-RLVR；LeapLabTHU/limit-of-RLVR
- 亲手实验（选题制）：从“合成数据自训练一轮” / “拒绝采样提升数学” / “self-play 生成题目”三者中选一个，做最小闭环 + 反证
- 判分证据：小实验 + 明确写出“这个方向的 no-go 是什么”（负结果不许擦除）
- 常见误解：把“前沿”当成必须立刻掌握（正确姿势是保持索引能力 + 每季度更新一次本模块）

1.12 模块 ↔ 课题 ↔ 周次 对照

模块	主课题	周次	讲（对标）	做（手写区）	证（判分/证据）
M0	前置/贯穿	W1	d2l ch2-5	micrograd + MLP	数值梯度对照
M1	课题02	W2	d2l ch8-10	bigram → RNN → Trans-former	bpb 对照表
M2	课题01	W1	CS336 A1 + HF ch2	手写 BPE	pytest 全绿 + 压缩率
M3	课题02	W2-W3	CS336 lec03-04 + rasbt ch3-4	手写 Trans-former	pytest + 改坏实验
M4	课题03/04/06	W3-W9	CS336 A2/A3 + nanochat	AdamW + sweep + DDP	拟合图 + 扩展效率

模块	主课题	周次	讲（对标）	做（手写区）	证（判分/证据）
M5	课题04/05	W5–W8	nanochat engine + CS336 lec10	KV cache + Triton kernel	三图 + 实测数字
M6	课题08	W10–W12	理由书 ch3–8 + RLHF Book	SFT/DPO/GPT-O grpo 对照	全绿 + 对照表
M7	课题07/08	W9–W12	Reasoning ch3 + CS336 lec12	判定器 + 评测	假阳性分析 + 误差棒
M8	课题09/10	W13–W14	李宏毅 Context Eng. + Agents MOOC	最小 ReAct + self-judge	轨迹分类 + 消融
M9	课题11（选题）	W15–W16	论文精读 + MOOC	选题小实验	no-go 记录

1.13 与课程（CORE100299）六板块的映射

课程板块	对应本图模块	课程大论文可用素材
绪论	M1	语言模型史一页
搜索求解	M8（Agent 的搜索/规划）	ReAct 与树搜索对照
知识表示	M8（记忆/工具）	context engineering 案例
机器学习（监督/深度/强化/无监督）	M0/M3/M4/M6	预训练+后训练全链路
智能体	M8	最小 ReAct + self-judge 消融
大语言模型	M2–M7	0.1B 端到端 + scaling law

2 对标教学资源清单（2026-09-17 核实）

目的：给”讲”这条腿找最好的老师。每个知识点都指定一份对标材料，由导师讲解 + 你复述 + 你动手，形成”三师制”：对标名师讲概念 / 我讲你的具体卡点 / 判分器讲对错。

核实纪律（承仓库铁律”引用任何数字先复现”）：本表”核实”列标 ☒ 的，是我在 2026-09-17 当日通过 gh api 或官网页面实际打开确认的；标 ☐ 的表示尚未亲自核实，不许作为唯一依据，用前先核。标 ☐ 的是已知坑（过时/收费/不可达）。

2.1 一、主线三件套（工程 + 判分，本学科骨架）

#	资源	类型	语言	覆盖模块	核实	为什么选它
1	Stanford CS336 (stanford-cs336/lectures + assignment1 - basics ... assignment5 - alignment)	课程 + 判分器	英	M2-M7	☒ commit 已锁	唯一一把「从零写」做成自动判分的课程；2026版 A5 已含 GRPO/GSPO/MaxRL

#	资源	类型	语言	覆盖模块	核实	为什么选它
2	karpathy/nanogpt (MIT)	金链路工程	英 8,087 ☆	M2-M5, M7	 commit 92 d63d4e8bb4	20 个文件 覆盖 tok- enizer→预 训 练→SFT→RL→评 测→推 理引 擎→工 具执行 ；单旋 钮 —— depth； 支持 CPU/MPS
3	jingyaogong/xinmin (Apache-2.0)	中文金链路 + RL 脚本	中 (61,450 ☆)	M2-M6		中文文 档 + train_grpo /ppo/ dpo/ agent.py ；单卡 3090 上”3 块 钱 2 小 时”；适 合读代 码学中 文术语

2.2 二、理论线的”对标名师”（补”讲”的深度）

#	资源	类型	语言	覆盖模块	核实	怎么用
4	rasbt 《Building a Reasoning Model (From Scratch)》 + rasbt/reasoning-from-scratch (5,000+☆, Apache-2.0)	14 + 逐章 note-book	英	M6/M7/M8		教学法范 本: ch3 先建评测再上任何技术 (“so every later improvement is measured rather than asserted”) ; ch5 self-refinement; ch6 GRPO 从零 ; ch2-4 可 CPU 跑, 只有 RL 章需要 GPU

#	资源	类型	语言	覆盖模 块	核实	怎么用
5	Nathan Lam- bert 《Reinforcement Learn- ing from Human Feed- back》 (rlhfbook.com, 免费在线 , 2026 版)	书 (免 费)	英	M6/M7	✓	后训练 理论主 线: 指 令微 调/奖励 模 型/PPO/DPO/RLVR/拒 绝采 样/合成 数据/过 度优 化/评测 , 17 章
6	李宏毅 (台大) GenAI- ML 2025 Fall / 机器学 习 2026 Spring	视频 + 讲义 PDF	中	M3/M6/M8	✓ 站点 已核	中文讲 得最清 楚的一 手: 含 《Context Engi- neer- ing》专 讲 (Agent 时代的 核心)

#	资源	类型	语言	覆盖模块	核实	怎么用
7	李沐《动手学深度学习》(zh.d2l.ai, B站 236 集)	书 + 视频 (免费)	中	M0/M1/M3	✓ I5	“不是通过幻灯片讲解，而是通过解读代码、动手调参跑实验来学习”——中文知行合一的原典
8	rasbt《Building a Large Language Model (From Scratch)》+ rasbt/LLMs-from-scratch (105,140 ☆, 2026-09-17 仍在更新)	note-book	英	M2-M4, M6	✓	代码级最细；附录含 GQA/MLA/SWA/MoE/KV cache/FLOPs 分析/LoRA, 正好是 M3/M5 的补充读物

#	资源	类型	语言	覆盖模块	核实	怎么用
9	HF LLM Course (13 章, ch11-12 含 LoRA/SFT/GRPO)	课程 (免费)	英	M2/M6	✓	由 transformers/TRL 库作者写, ch8 「调试训练流水线」是别处没有的
10	HF Agents Course (4 units + 最终项目排行榜 + function-calling 微调 bonus)	课程 (免费, 有证书)	英	M8	✓	三个框架并学 (smolagents/LlamaIndex/LangGraph), 最后提交到公开排行榜——自带客观验收
11	Berkeley LLM Agents MOOC / Agentic AI MOOC (rdi.berkeley.edu、agenticai-learning.org)	公开课录像 + slides	英	M8/M9	✓	有一讲就叫“Post-Training Verifiable Agents”、一讲“Agent Evaluation”——正是唐杰作业⑤的前沿坐标

#	资源	类型	语言	覆盖模 块	核实	怎么用
12	Karpathy 视频 Neural Net- works: Zero to Hero 》 (micrograd/makemore/nanoGPT)	视频	英	M0/M1/M3	系列 存在性 已核	直觉地 基 ； micrograd 是 M0 手写实 验的模 板（ 但你要 自己 写）
13	3Blue1Brown 神经网络 系列	动画视 频	英（有 中字）	M0/M3	⌚	视觉直 觉；用 来预热 atten- tion/反 向传播
14	HF Ultra- Scale Play- book	长文（ 免费）	英	M4	⌚	分布式 训练 （DDP/FSDP/TP/PP） 的工程 全景 ， W9 前核实
15	《大规 模语言 模型： 从理论 到实践 》（复 旦 张奇 等）	书	中	M2-M6	⌚	中文教 材备选 ； 与 mini- mind 配 套读

2.3 三、评测与环境类（M7/M8 的判分件）

#	资源	覆盖	核实	用途
16	harbor— framework/ terminal— bench (+ 冻 结版 terminal— bench—1)	M8	✓	长程 Agent 的连续基准 (v2 新仓 , v1 已冻结 改名)
17	SWE—bench/ SWE—bench 、SWE— bench/SWE— smith	M7/M8	✓	代码任务评 测 / 合成可 验证环境
18	GSM8K / MATH-500	M7	✓ (在 rasbt ch3 用 作 verifier 对 象)	数学可验证 奖励的标准 件
19	PrimeIntellect —ai/verifiers (4,626☆, MIT)	M7/M8	✓ commit 已锁	RL 环境 + 评测的抽象 层, 做课 题09 的参照
20	EleutherAI/lm —evaluation —harness 、UKGovernmentBEIS /inspect_ai	M7	✓	学术评测标 准件 / harness 设计 参考
21	walkinglabs/ hands—on— modern—rl (4,395☆, 中文)	M6	✓	RL→RLVR→Agentic RL 中文教材 , 实验标注 CPU/xGPU 且挂免费在 线 Notebook
22	McGill—NLP/ nano—aha— moment	M6	✓	单文件单卡 从零 GRPO (阅 读用, 不宜 作依赖)

2.4 四、明确标注的坑（用前必读）

资源	坑
CS336 A3	需要 <code>A3_API_KEY = 8</code> 位学号，非斯坦福在校生拿不到 → 本学科用自建 <code>sweep</code> 替代（课题03 已按此设计）
CS336 A2	<code>flash-attn</code> 安装特殊： <code>uv sync --no-install-package flash-attn</code> 再 <code>uv sync</code>
CS336 A5	2026 版换成 <code>Olmo-2-1B @ GSM8K</code> ，且用 <code>Modal</code> 云函数 → 显存要求最高，放在 <code>W12</code> 且用双 <code>T4</code>
Kaggle P100	若分配到 <code>P100 (SM60)</code> ， Triton 不可用（需 <code>CC ≥ 7.0</code> ）→ 课题05 必须重开会话直到拿到 <code>T4</code>
<code>karpathy/nanoGPT</code> 、 <code>build-nanogpt</code> 、 <code>llm.c</code>	已被 <code>nanochat</code> 取代或停滞，只作历史读物
<code>karpathy/LLM101n</code>	已归档（ archived ），且 <code>Eureka Labs</code> 至今未完整发布 → 只当索引，别当路线
HuggingFace FineWeb	我用 <code>gh api</code> 查 <code>HuggingFaceFW/fineweb</code> 、 <code>huggingface/fineweb</code> 均 404 → 不要写这两个路径，语料走 <code>TinyStories / stanford-cs336/owt-sample / dolma</code>
<code>terminal-bench</code>	已改名分仓（ <code>v1</code> → <code>terminal-bench-1</code> ， <code>v2</code> → 新仓），旧链接会 404

2.5 五、三师制使用方法（每个知识点的标准动作）

- ① 先读对标材料的那一节（不超过 25 分钟，带着“我要回答哪 3 个问题”去读）
- ② 合上材料，用自己的话讲给我听（我按 `M0-M9` 的“核心问题”追问，专挑你含糊的地方）
- ③ 我讲你的卡点（只讲你卡住的那一层，不重讲整章）
- ④ 上手写（手写区任务，最小可运行）
- ⑤ 跑判分（`pytest` / 自建判定器）→ 出证据

⑥ 改坏它 → 预测 → 验证 → 修好（第3条硬标准）

分工原则：对标名师负责“标准讲法”，我负责“你的卡点 + 判分 + 记忆归档”，任何一方都不替代你动手（见《造轮子边界.md》）。

3 知行合一规程（本学科教学法核心）

立规 2026-09-17。缘起：用户指出”实操性够强，但知识讲解偏少”，并明确要求把知行合一贯穿全流程。本规程定义：什么算”学过”，什么算”学会”，以及每一层怎么被强制咬合。

一句话：本学科没有”听懂了”这个状态，只有在代码里被验证过的”懂了”。

3.1 一、把”知行合一”变成可执行的四件咬合

咬合件	名称	谁的活	不通过的后果
讲	导师讲 + 对标名师	我（ ≤ 20 分钟一块， ≤ 150 字一个概念块）	讲完必须立刻出”代码级提问”
做	手写区最小可运行	你	没跑起来 = 这一课不算上
证	判分器 / 实验证据	我搭台，你跑	无证据的结论一律不采纳
反	改坏实验 + 打脸记录	你预测，你验证	只会正确写法的人，遇到 bug 一定卡死

四件缺一 → 本知识点标  未通过，不进 PROGRESS 的 .

3.2 二、知识卡规范（每个知识点一张，落 知识卡/ 或课题目录）

```
# 知识卡 · <知识点名>（模块 M?）
- 一句话定义：
- 直觉（给别人讲会用的话）：
- 公式/伪代码骨架（手写，不许贴代码）：
- 对标出处：<材料名 + 章节>
- 上手写实验：<文件路径 + 判分命令>
- 判分证据：<pytest 名 / 图 / 数字>
- 改坏实验：改哪里 → 你预测的现象 → 实测现象
- 常见误解：
```

- 记忆锚（隐喻+口诀+符号位置）：
- 间隔重复卡号：

知识卡由你口述、我整理（你说我写），但”直觉”“改坏实验的预测”两栏必须是你自己的话。若某栏你说不出来 → 这节课没上完。

3.3 三、讲课节拍（每周两个时段，各 1.5–3h）

时段 A（理论）：讲（≤20min 一块）→ 你复述 → 我挑含糊处追问 → 你写知识卡 → 出代码级提问
 时段 B（实验）：读对标材料对应节 → 你写手写区 → 跑判分 → 记录证据 → 改坏实验 → 归档

硬纪律：- 一次讲透，不挤牙膏（沿用你已定的节奏：你多讲、我多写；批量 3–5 个关键问题）。- 每个概念块 ≤150 字，讲完必须配一个”立刻能跑的最小实验”。- 讲课时不许给手写区代码（见《造轮子边界.md》）；给的是”该去哪里找、该测什么”。

3.4 四、代码级提问（“知”与”行”的咬合点，本学科独创）

每讲完一块，我出 3–5 道代码级提问，形式固定为”改坏它 / 预测现象 / 给出诊断”：

类型	例子
改坏预测	“把 attention 的因果 mask 注释掉，loss 会怎样？训练 100 步内你会看到什么现象？”
放大器	“把 RoPE 换成可学习位置嵌入，在训练/外推两种场景下各自会怎么退化？”
诊断	“你观察到 loss 在第 400 步突然变 NaN，请按顺序列出你会检查的 5 个量。”
代价	“把词表从 32k 扩到 128k，参数量、序列长度、每 token 成本各变多少？（估数量级）”

类型	例子
反事实	“如果 DDP 改成 FSDP，同样的 batch，显存和速度分别会怎么动？为什么？”

判分标准：预测错不算失败（预测错 + 跑实验打脸 = 最高质量的学习），但“没有预测就跑”算失败，因为那等于放弃了知行之间的连接。

3.5 五、“真的懂了”的三条硬标准（与《知识地图》一致，此处为验收动作）

1. 能讲清 → 你对着外行话讲一遍，我专门装不懂追问三层（Why → Why not → What if）。
2. 能指出代码位置 → 你在我画出的 手写/ 或上游代码里，不搜索直接指出对应函数/行号区间。
3. 能改坏并修好 → 我在你没看过的地方埋一个 bug（或你自己改坏），你在不看不该看的代码的前提下，用实验定位并修好；修好过程必须留日志进 证据/。

三条全过才写 ☒；只过第 1 条 = 会考试，不会干活。

3.6 六、反讲机制（费曼 + 红队）

- 每模块收口：你当老师，我当刺头学生，我只问“为什么”，不纠正、只追问，直到你卡住或讲穿。
- 卡住的地方就是下一个知识点的入口，直接写进 HANDOFF.md 的“下一步”。
- 结项前：你来当评审，对着自己的报告做 3 条自我攻击（“这个结论最可能错在哪？”），与导师审稿意见并列存档 → 这就是唐杰作业⑦”问题/动机/方法/结果一项都不能糊”的本地化实现。

3.7 七、与其它章程的关系

- 谁写代码 → 《造轮子边界.md》（R1-R10 手写 / B1-B8 代写 / C 协作）
- 讲什么、讲多深 → 《知识地图-大模型全景.md》（M0-M9 四件咬合件）

- 用哪份材料讲 → 《对标教学资源清单.md》（三师制）
- 怎么算过 → 《判分与验收标准.md》（三层判分 + 每课题通过证据）
- 什么时候讲 → 《实训路线图.md》“知识讲解锚点”列

4 实训路线图 · 16 周 × 11 课题 × 唐杰 7 条 × CS336 映射

2026-09-17 并入 人工智能基础与应用/（实训轨）：本学科三轨合一——课程轨（CORE100299 课堂事务，见../课程轨/课程进度.md）、实训轨（本文件 + 11 个课题）、知识轨（知识地图-大模型全景.md M0-M9）。配套：《对标教学资源清单.md》（跟谁学）、《知行合一规程.md》（什么叫学过）、《造轮子边界.md》（谁写代码）、《算力与成本预算.md》（花多少）、《判分与验收标准.md》（怎么算过）。

立图 2026-09-17。基准：第 1 教学周 = **2026-09-14**（与校历、与唐杰清华开课同周）。节奏：16 周跟课（每周 5-6h，约 2 个学习时段）。预算：零现金，只用免费算力（见《算力与成本预算.md》）。

4.1 一、目标（一句话）

用零现金预算，把唐杰 2026 秋季作业清单一条不落地走完，每一环都留下可复现证据、可执行判分与一页报告；最终产出一份 NeurIPS 格式的英文技术报告 + 一个可现场 demo 的作品。

4.2 二、唐杰 7 条 ↔ 本学科课题 ↔ 判分来源（主映射表）

唐杰作业条目	对应课题	官方规格 / 判分器	算力档
① 从零写 Tokenizer + Transformer，端到端训 0.1B	课题01、02、04	CS336 A1 (tests/)	T0 → T2
② 手写 Triton attention kernel，测多卡训练/推理增益	课题05、06	CS336 A2	T1 / T2
③ raw dump 洗语料，拟合 scaling law 再外推	课题07、03	CS336 A4 + A3 (A3 需自建替代)	T0 / T1
④ 同一基座做 SFT / DPO / RLVR 对照	课题08	CS336 A5 (含 tests/test_grpo.py)	T2

唐杰作业条目	对应课题	官方规格 / 判分器	算力档
⑤ 可验证环境 + harness, 训长程 Agent, 鼓励 self-judge loop	课题09、10	无官方判分器 (前沿区) → verifiers / terminal-bench 思路	T2
⑥ 2-3 人组队, 英文 NeurIPS 格式, W16 现场 demo	课题11	本仓库报告协议 + LaTeX 模板	—
⑦ 作业 40% / 大项目 60%; 问题/动机/方法/结果一项都不能糊	全学科判分口径	《判分与验收标准.md》	—

4.3 三、16 周课表 (周次 / 课题 / 交付物 / GPU 需求)

周	日期	课题	本周交付物 (硬指标)	GPU
W1	09-14→09-20	课题01 BPE 分词器	手写 BPE + test_tokenizer.py /test_train_bpe.py 全绿 + 压缩率表	0
W2	09-21→09-27	课题02 Transformer 骨架	attention/MLP/RoPE/RMSNorm 手写 + 形状与数值测试过	0
W3	09-28→10-04	课题02 收口 (国庆周, 减半)	训练循环 + AdamW + 序列化测试过; loss 稳定下降曲线	0

周	日期	课题	本周交付物 (硬指标)	GPU
W4	10-05→10-11	课题03 Scaling Law 预演	5 点小规模 sweep + 拟 合 + 外推预 测 + 打脸记 录	0 (CPU)
W5	10-12→10-18	课题04 0.1B 端到端 (上)	数据管线 + 双 T4 冒烟 + 首轮训练启 动 (断点续 训可用)	6- 9h (2×T4)
W6	10-19→10-25	课题04 收口	bpb/CORE 指标 + 采样 样本 + 实测 成本账单 vs 估算	6- 12h (2×T4)
W7	10-26→11-01	课题05 Triton kernel (上)	naive 实现 → 分块实现, 正确性对齐	4h (1×T4)
W8	11-02→11-08	课题05 收口 + 课题06 开 题	speedup / 显 存 / profiling 三张图 (含 硬件口径声 明)	6h (2×T4)
W9	11-09→11-15	课题06 多卡 与分布式	DDP vs FSDP 对照 + 扩展效率 曲线 + 通信 占比分析	8h (2×T4)
W10	11-16→11-22	课题07 数据 清洗与去重	过滤/去重规 则 + 有无清 洗的对照组 loss 对比	6h (1×T4)
W11	11-23→11-29	课题08 SFT	SFT 基线 + 与基座对照	8h (2×T4)

周	日期	课题	本周交付物 (硬指标)	GPU
W12	11-30→12-06	课题08 DPO + RLVR	SFT / DPO / GRPO 三方 对照表 (同 基座、同评 测口径)	10h (2×T4)
W13	12-07→12-13	课题09 可验 证环境 + harness	1 个自建可 验证环境 + rollout 成功 率 + 判定逻 辑单测	6h (2×T4)
W14	12-14→12-20	课题10 长程 Agent + self-judge	多步任务串 联 + self-judge 有 无的消融	8h (2×T4)
W15	12-21→12-27	蓄水周	补跑欠账 + 英文报告初 稿	4h (弹性)
W16	12-28→01-03	课题11 结项	NeurIPS 格 式英文报告 + 现场 demo + 总复盘	4h (弹性)

GPU 总需求 ≈ 97 小时，免费额度可用 ≈ 480 小时 (Kaggle 30h/周 $\times$ 16 周) $\rightarrow$ 安全系数约 $5\times$ (见预算章程)。

4.3.1 附：知识讲解锚点（每周讲哪个模块——补上”讲”这条腿，2026-09-17 新增）

每个知识点的”四件咬合”（讲/做/证/反）见《知行合一规程.md》；模块内容见《知识地图-大模型全景.md》；对标材料见《对标教学资源清单.md》。

周	讲解模块	本周要讲透的核心问题 (≤5 个)	配套材料
W1	M0 + M2	计算图与链式法则；熵/交叉熵/困惑度；BPE 为什么字节起步	d2l ch2–5；CS336 A1 handout
W2	M1 + M3 (上)	从 n-gram 到 attention 的动机；attention 在算什么；为什么多头	d2l ch8–10；CS336 lec01/03
W3	M3 (下)	RoPE vs 可学习位置；RMSNorm/pre-norm 为何关键；残差防什么	CS336 lec03–04；rasbt ch3–4
W4	M4 (上)	优化器与 LR 调度；混合精度；scaling law 如何指导算力分配	CS336 lec09/11；nanochat optim.py
W5	M4 (下) + M5 (上)	数据配比与去重；吞吐/显存/FLOPs 记账	CS336 lec05/13–14；nanochat base_train.py
W6	M5 (下)	KV cache 省了什么；采样策略；bpb/CORE 指标口径	nanochat engine.py/core_eval.py ；CS336 lec10/12
W7	M5 (续·kernel)	GPU 内存层级；Triton 编程模型；flash attention 的分块与 online softmax	CS336 lec05–06；triton 官方教程
W8	M4 (续·并行)	DDP/FSDP/TP/PPCS336 的取舍；为什么不是线性加速	lec07–08；nanochat optim.py

周	讲解模块	本周要讲透的核心问题 (≤5 个)	配套材料
W9	M7 (上)	评测口径、污染、方差、误差棒；“变好了”如何被证明	rasbt ch3; CS336 lec12
W10	M7 (下) + M6 (上)	过滤/去重的判定规则；指令微调与损失掩码	CS336 A4; 《RLHF Book》ch4
W11	M6 (中)	偏好数据、奖励模型、DPO 的隐式奖励	《RLHF Book》ch5/8/11; CS336 A5
W12	M6 (下)	GRPO 的组内优势、KL 正则、reward hacking、过度优化	rasbt ch6-7; 《RLHF Book》ch6/7/14
W13	M8 (上)	工具调用/ReAct/context engineering; harness 为什么决定成败	李宏毅《Context Engineering》 ; Agents MOOC
W14	M8 (下)	长程任务的误差累积; self-judge 何时有效、何时有害	rasbt ch5; Berkeley Agentic AI MOOC
W15-16	M9 (选题)	前沿采样：MoE/长上下文/合成数据闭环; no-go 的写法	论文精读清单 ; awesome- RLVR

4.4 四、11 课题总表 (手写要点 / 判分来源 / 通过证据)

#	课题	手写区核心 (不可代写)	判分来源	通过证据
01	BPE 分词器	merge 循环、特殊 token、bytes 回退、流式编码	CS336 A1 tests/ test_tokenizer.py 、test_train_bpe.py	pytest 全绿 + 压缩率表
02	Transformer 骨架	attention、RoPE、LayerNorm、SwiGLU、残差、初始化	CS336 A1 test_model.py 、test_nn_utils.py	pytest 全绿 + 参数量核对
02b	训练循环与优化器	AdamW/Muon 更新式、LR schedule、梯度裁剪、精度管理	CS336 A1 test_optimizer.py 、test_serialization.py	pytest 全绿 + loss 曲线
03	Scaling Law	数据采集脚本口径、拟合、外推、打脸记录	自建判分：预测 vs 实测偏差	5 点 sweep 数据 + 拟合图 + 外推预测存档
04	0.1B 端到端	(复用 01/02 产物) 预训练编排、断点续训判断	自建判分：loss/bpb 达标 + 采样可读	bpb/CORE + 采样样本 + 成本账单
05	Triton attention kernel	kernel 本体 (分块、online softmax)	CS336 A2 + 正确性对齐参考实现	三图 (speedup/显存/profile) + 数值误差记录
06	多卡与分布式	并行策略选择、通信原语、扩展效率分析	CS336 A2 分布式部分 + 自建 benchmark	扩展效率曲线 + 通信占比

#	课题	手写区核心 (不可代写)	判分来源	通过证据
07	数据清洗与去重	过滤/去重规则的判定逻辑	CS336 A4 + 自建消融	规则清单 + 对照组 loss 差
08	SFT / DPO / RLVR	损失函数与优势估计 (A/group norm/重要性比)	CS336 A5 tests/ test_grpo.py + 自建对照	三方对照表 (同口径)
09	可验证环境	判定逻辑 (verifier/reward)	自建单测 + auto 成功率	判定逻辑测试 + 成功率 + 失败样本分析
10	长程 Agent	self-judge 协议设计	自建消融	有无 self-judge 的成功率差 + 案例
11	结项	报告骨架与结论判断	报告协议 + 导师审稿	英文报告 + demo

4.5 五、规模三档（零预算版）

档	硬件	模型规模	数据	用途
T0	沙箱 CPU (2 vCPU/3GB) 或 Colab CPU	1–10M	TinyStories 子集 / owt-sample 采样	课题01–03 的全部正确性与判分
T1	Kaggle 1×T4 16GB	0.03–0.1B	TinyStories 全量 + 子集	课题05/07 的单卡实验
T2	Kaggle 2×T4 (32GB 合计)	0.1B	同上 + 合成任务数据	课题04/06/08/09/10

规模下探是主动取舍，不是妥协：把”0.1B”从口号变成每次会话都能真跑的实验，代价是学不到万卡级工程；收益是 16 周内全链路闭环。前沿规模直觉另走 论文精读/ 补齐。

4.6 六、跨学科搭桥（讲课时主动用）

- RL 的目标函数 $\rightarrow$ probability/ 的期望、方差、条件期望（REINFORCE = 用样本估期望）
- 语言模型的困惑度/交叉熵 $\rightarrow$ probability/ 的信息量直觉
- scaling law 的幂律拟合 $\rightarrow$ probability/ 的最小二乘与回归
- 结项报告 $\rightarrow$ 论文精读/ 的论文写作规范；demo $\rightarrow$ 人工智能基础与应用/ 的大论文素材

5 造轮子边界 · 谁写代码（本学科最高契约）

立约 2026-09-17，用户拍板口径：“分层双轨”。本学科同时要求两件互相冲突的事：① 唐杰作业要求”不会动手、只会调 API 的先下去”；② 你的时间是稀缺资源，16 周内跑完全链路。解决办法不是折中，而是划死边界：学习目标必须你手写；非学习目标必须我代写。边界之下，任何一方越界都算事故。

5.1 一、三条铁律

- 1. 手写区（课题NN/手写/）：**Agent** 禁止写入任何文件。不得给代码、不得给伪代码、不得把第三方实现（含 nanochat / CS336 参考实现 / GitHub 上的答案库）当答案指给你。允许的动作只有五类：解释报错、解释概念、审查你已写的 diff、建议 sanity check / toy example / profiler 检查、指路官方文档与讲义。
- 2. 脚手架区（课题NN/脚手架/、环境/）：**Agent** 全权负责且必须主动完成，不等你催。这部分写一遍就够，重复劳动零学习收益。
- 3. 越界的正确姿势：你说”给我看答案”→ 走提示阶梯（五级）→ 到第五级给答案后，必须挂近迁移重考卡片（隔 1-3 天、无提示、换场景再考）。给了答案不挂卡片 = 违规。

5.2 二、手写清单（学习目标，共 10 项）

编号	手写内容	为什么必须亲手写
R1	BPE 训练与编码算法 (merge 循环、特殊 token、bytes 回退、流式编码)	分词决定了模型看到的世界；面试/科研高频
R2	Transformer 全部组件 (attention、RoPE、RMSNorm、LayerNorm、残差、初始化)	梯度与形状直觉只从亲手写里长出来
R3	训练循环与优化器 (AdamW/Muon 更新式、LR schedule、梯度裁剪、精度管理)	训练不收敛时，只有写过的人会 debug
R4	反向传播验证 (gradient check、数值梯度对照)	这是”我懂了”与”我能跑”的分界线

编号	手写内容	为什么必须亲手写
R5	Triton attention kernel 本体（分块、online softmax）	唐杰作业②的核心；性能直觉的唯一来源
R6	Scaling law 拟合与外推（数据采集口径、拟合、预测、打脸记录）	这是”科研判断力”的训练区，不能外包
R7	SFT / DPO / RLVR 的损失函数与优势估计（含 group normalization、重要性比）	后训练是 2026 的重心；抄公式等于没学
R8	数据过滤/去重的判定规则（启发式规则设计）	数据决定上限；规则设计的品味靠自己做
R9	可验证环境的判定逻辑（verifier / reward function）	唐杰作业⑤的技术核心
R10	self-judge loop 的协议设计（何时判、判什么、如何防退化）	唐杰课明确鼓励的方向；前沿无标准答案

5.3 三、代写清单（脚手架，Agent 负责，共 8 项）

编号	代写内容	交付形态
B1	环境与依赖（uv / CUDA / 镜像 / 引导脚本）	环境/bootstrap*.sh + uv.lock
B2	数据下载/解压/分片/上传（含 Kaggle Dataset 打包）	环境/数据/ 脚本 + 清单
B3	日志、曲线绘图、实验编排、断点续训封装	课题NN/脚手架/ 脚本
B4	判分脚本（拉取 CS336 官方 tests/ 并接入适配器）	环境/judge/ + judge.sh

编号	代写内容	交付形态
B5	证据带自动归档 (run.yaml / 日志 / 曲线 / 产物哈希)	课题NN/证据/ 由脚本生成
B6	报告模板与排版 (Markdown + LaTeX/NeurIPS 模板)	环境/报告模板/
B7	配额与超时治理 (会话预检、超时自停、配额账本)	环境/预检.md + 脚本
B8	checkpoint 上传/下载与版本管理	课题NN/脚手架/ckpt_*.sh

5.4 四、协作区（一起定，你拍板）

编号	事项	分工
C1	架构与底座选型、上游引入范围	Agent 出方案与证据 → 你拍板
C2	超参搜索空间、指标口径 (bpb / CORE / 通过率)	Agent 出方案 → 你定口径
C3	实验优先级与是否值得重跑	Agent 给成本估算 → 你决定
C4	可验证环境的题目设计	你出设计意图 → Agent 评估可行性与成本

5.5 五、违规示例（判例，会长大）

- **E-01**（预防性记录）：Agent 往 手写/ 里写入实现文件 —— 即使内容正确也算违规，必须改为“审查意见 + 测试建议”落到 判卷记录.md。
- **E-02**（预防性记录）：Agent 说“你可以参考 nanochat 的 flash_attention.py 第 30 行” —— 违规。正确做法：说明该文件在整体架构中的位置与接口契约，指出官方 Triton 教程与 CS336 A2 handout 的对应章节。
- **E-03**（预防性记录）：Agent 帮你把实验卡写好之后直接开始烧 GPU 时间 —— 违规。实验卡必须由你确认（哪怕只是回一句“跑”）。

5.6 六、边界自检（每次动手前问一遍）

1. 我现在要写的这个文件，属于 R1-R10 还是 B1-B8？
2. 如果是 R 类 → 我只能读、只能评论，不能写。
3. 如果是 B 类 → 我必须写，且要写完整（不留 TODO 给用户）。
4. 如果两边都不是 → 进协作区，先出方案再动手。

6 AI 使用红线（本学科版）

改编自 **Stanford CS336** 官方 AGENTS.md (stanford-cs336/assignment1-basics, 2026-09-17 读取原文)，按用户的“分层双轨”口径改写。CS336 原文的立场是“Teaching Assistant, Not Solution Generator”，本学科的立场是“学习目标区=TA，基建区=工程师”。

6.1 一、最高原则

学习目标（R1–R10）必须由用户亲手完成；非学习目标（B1–B8）必须由 **Agent** 完成。边界见《造轮子边界.md》。任何一端的越界都算事故，写进 memory/ERRORS.md。

6.2 二、手写区：Agent 应当做（SHOULD）

- 解释概念：只讲用户卡住的那一层，不重讲整章；用一个反例或一个玩具例子收尾。
- 指路：指出对应讲义章节（CS336 / d2l / 李宏毅 / rasbt / RLHF Book 的具体哪一节）与官方文档。
- 审查用户代码：提 3–5 条意见，指向“哪个不变量可能被破坏”“哪个边界情况没测”，不直接给修好的代码。
- 帮助 debug：问引导性问题（“你检查过 mask 施加在 softmax 之前还是之后吗？”），而不是给修复。
- 解释报错：Python / PyTorch / CUDA / Triton / 分布式栈的报错含义与排查顺序。
- 建议验证手段：形状断言、玩具输入、profiler 检查、消融、梯度检查（数值对照）。

6.3 三、手写区：Agent 禁止做（SHOULD NOT，照抄 CS336 并加严）

- ✗ 写任何 Python 或伪代码
- ✗ 给出问题的解法或思路解法
- ✗ 完成手写区里的 TODO
- ✗ 编辑用户仓库里的手写区文件
- ✗ 大段重构用户代码变成“成品答案”
- ✗ 把作业要求直接翻译成可运行代码
- ✗ 替用户实现核心组件（tokenizer、transformer 块、优化器、训练循环、Triton kernel、分布式逻辑、scaling law 管线、数据过滤/去重、对齐/RL 方法）

- **✗** 把第三方实现当答案指路（CS336 原文：课程材料自包含）——本仓加严：包括 nanochat / minimind / GitHub 上的作业答案库。可以指出“该功能在架构中的位置与接口契约”，但不得指出“去抄哪几行”。

6.4 四、脚手架区：Agent 应当做（且必须做完）

- **✓** 环境/：依赖锁、引导脚本、会话预检、报告模板、数据打包
- **✓** 脚手架/：实验编排、日志、曲线绘图、断点续训、checkpoint 管理、判分脚本、证据归档
- **✓** 课程轨/：课程进度台账、大论文模板与格式整理
- **✓** memory/：HANDOFF / PROGRESS / ERRORS / LEARNINGS / MEMORY / drill-ledger 的维护

判据：脚手架代码里不允许留 **TODO** 给用户——那是把学习目标混进了基建。

6.5 五、例外与补偿（本学科特有，CS336 没有）

情形	允许	补偿动作
用户明说”卡死了，给我看答案”	走提示阶梯五级，第 5 级给答案	必须挂”近迁移重考”卡（隔 1-3 天、无提示、换场景复考）
用户连续 2 轮在同一处打转	由我提议”降档”：给一个更小的可跑通示例（不是我写他的目标代码）	记录到 ERRORS.md，下一课换讲法
环境/工具链阻塞（非学习目标）	直接由我修好，不占用用户时间	记入 证据/ 的环境变更

6.6 六、教学法提示（照抄 CS336 的方法论，本学科已内化）

用户求助时，按此顺序：①问澄清问题（试过什么/期望什么/发生了什么）→ ②引用讲义概念而非直接回答 → ③建议下一步而不是替他做 → ④审查代码并指出可能 **bug**/缺失检查的方向性提示 → ⑤解释”为什么”而不只是”怎么做” → ⑥优先用测试与不变量而非修复。

6.7 七、学术诚信与”知行合一”的关系（本学科的立场）

CS336：AI 只能用于低层次编程问题与高层次概念问题，不得直接解题；“我们强烈建议在完成作业时关掉 AI 自动补全（Cursor Tab / Copilot）——

我们发现它让人更难深入内容。”

本学科采纳这条建议的一半：动手写 R1–R10 时，手写区文件的编辑器内 **AI** 自动补全也要关（或在 脚手架/ 里明确标注禁止生成手写区文件）。理由：本学科的验收标准是“能改坏并修好”——如果代码是补全出来的，你就没有能力在它坏掉时诊断它。

6.8 八、越界举报（自检清单，每轮自问）

1. 我这一轮写的文件，属于 R 类还是 B 类？
2. 我给的是“问题”还是“答案”？（给答案前是否走了阶梯、是否挂了重考卡？）
3. 我有没有指路第三方实现？
4. 用户这一轮有没有“预测 → 验证”的环节？没有的话，我是不是又变成讲解器了？

7 算力与成本预算（零预算版）

立规 2026-09-17。用户拍板：零现金预算，但要求全量对标唐杰 7 条（含多卡与长程 Agent）。本文件证明这个组合可行，并定义它的代价与纪律。

7.1 一、免费资源池（2026-09-17 核实）

平台	免费额度	硬件	会话上限	持久存储	卡要求	角色
Kaggle	30 GPU·小时/周（每周重置）	2×T4（32GB 合计）或 1×P100	12h（支持后台运行、每 5 分钟自动存档）	20GB /kaggle/working	无需	主战场（唯一能做”多卡”的免费档）
Google Colab 免费	~15-30 GPU·h/周（未公布，随需求波动）	1×T4 16GB	12h，可能提前断	Google Drive（易失）	无需	备用（短实验、CPU 实验）
沙箱 CPU（本仓）	无限	2 vCPU / 3GB RAM	—	仓库（注意体积纪律）	无需	T0 全部课题的正确性与判分
Lightning AI 免费档	约 15 credits/月	T4/A10G 等	~3-4h 重启	50GB	无需	备用（额度与 GPU 型号未核实，用前先验）
SageMaker Studio Lab	4h/次、4h/24h	1×T4	4h	15GB	需账号	极端备用

平台	免费额度	硬件	会话上限	持久存储	卡要求	角色
Modal	\$30/月 免费额度 (CS336 赞助商 , B200 \$6.25/h ≈ 4.8h/月)	B200	按任务	卷	需国际 支付/网 络	可选升 级, 非 计划内

结论：16 周可用 ≈ 480 GPU·小时（Kaggle 30h $\times$ 16 周），本学科计划只用 ≈ 97 小时 $\rightarrow$ 安全系数约 $5\times$ 。

7.2 二、三档规模（零预算下的能力边界）

档	硬件	模型规模	典型任务	单次时长
T0	沙箱 CPU / Colab CPU	1–10M	课题01–03 全部、判分 器、知识卡 实验	分钟级
T1	Kaggle 1 $\times$ T4 16GB	0.03–0.1B	课题05 Triton bench、课 题07 数据消 融	1–4h
T2	Kaggle 2 $\times$ T4 32GB	0.1B	课题04 端到 端、课题06 DDP、课 题08 RL、课 题09/10 Agent	6–12h

7.3 三、成本估算（标记为估算，必须在课题04 用实测替换）

- 参考点（实测自 nanochat 公开文档）： $d12 \approx \text{GPT-1 量级} \approx 0.1\text{B}$,

在 8×H100 上约 6 分钟。

- T4 是 **SM75**，无 **bf16**（nanocat 自动退化 fp32，需手动 fp16 + GradScaler 才能提速）；等效算力约为 H100 的 1/15–1/25。
- 估算：单 T4 约 12–20h，双 T4 DDP 约 **7–12h**（扩展效率按 60–80% 估）。
- 执行方式：拆成两个 9h 会话（Kaggle 会话上限 9–12h），中间用 / kaggle/working 的 checkpoint 续训。
- △ 这个数字不许照抄进报告：课题04 结束后，用实测耗时替换，并在报告里写明“估算 vs 实测”的偏差。

7.4 四、每周配额分配（16 周）

周	课题	计划 GPU 用量	备注
W1–W4	课题01–03	0 h	纯 CPU，零消耗期（先把正确性做对）
W5	课题04（上）	6–9 h	数据管线 + 双 T4 冒烟 + 首轮训练
W6	课题04（下）	6–12 h	收口 + 采样 + 成本账单
W7	课题05（上）	4 h	单 T4（必须是 T4 ， P100 不可用）
W8	课题05（下）+ 课题06 开题	6 h	Triton bench 三图
W9	课题06	8 h	双 T4 DDP/FSDP 对照
W10	课题07	6 h	单 T4 数据消融
W11	课题08（SFT）	8 h	双 T4
W12	课题08（DPO+RLVR）	10 h	双 T4，最贵的一周
W13	课题09	6 h	双 T4 rollout
W14	课题10	8 h	双 T4
W15–W16	蓄水/补跑/结项	4+4 h	弹性
合计		≈97 h / 480 h	余量用于重跑与失败实验

7.5 五、硬纪律（出过事故的领域）

1. 实验卡制度：上 GPU 前必须先写实验卡，经用户确认才开跑。模板：

```
### 实验卡 #NN（课题NN / 模块 M？）
– 假设（一句话，可被证伪）：
– 命令（完整可复制）：
– 预算上限：≤ ? GPU·小时（默认 3）
– 停止条件：loss 不降 / 超过预算 / 出现 NaN → 立即停
```

- 成功判据（跑之前先写死）：
- 预期现象（**先写预测**，跑完对照——这是知行合一的咬合点）：
- 产物归档：证据/run.yaml + 日志 + 曲线 + 哈希

2. 会话预检（每次开 Kaggle 先跑）：nvidia-smi 看型号 → T4 或 2×T4 才继续；P100 立即重开会话（SM60, Triton 不可用，课题05 会在上面白烧配额）。
3. 断点续训是强制项：Kaggle 会话随时可能断；每 N 步写 checkpoint 到 /kaggle/working（20GB 持久）。
4. 超时自停：脚本内自带 wall-clock 上限（默认 3h），到点保存退出，不靠人盯。
5. 配额账本：跑完立刻在 memory/PROGRESS.md 记：日期 / 平台 / GPU 型号 / 实际耗时 / 累计消耗 / 本周余量。
6. 不做无判据的实验：没有“成功判据”就不许开跑——这是防止“烧配额摸鱼”的唯一有效办法。

7.6 六、可选升级路径（仅在零预算实在卡住时启动，需用户拍板）

方案	价格（2026 公开比价，下单前以平台现价为准）	何时值得
国内云 4090 按量	¥1.45–2.0/小时（AutoDL / 恒源云 / 智星云等）	双 T4 跑不动 0.1B RL (W12) 时；约 ¥30 可买 15h
Modal 免费额度	\$30/月 (≈4.8h B200)	需要 bf16 + 快迭代时（需国际支付）
Colab Pro	订阅制	不建议（免费档足够本学科用量）

诚实声明：本学科的设计前提是“免费额度足够”，依据是 §三 的估算与 §四 的配额表。若课题04 实测发现双 T4 需要 >20h，则触发“规模下探一级”（d12 → d10，或语料减半），而不是自动升级付费；升级必须由用户单独拍板。

8 判分与验收标准

立标 2026-09-17。继承仓库总纲 skills/科研式学习导师/SKILL.md §5a 《命题与判卷纪律》，并按本学科“可执行判分”的特性做加法。与总纲冲突处，以本文件的更严条款为准。

8.1 一、三层判分（每一层都必须留下原始输出）

层	谁判	判据	归档
L1 自动判分	判分器 (pytest / 自建 script)	通过 / 失败 / 覆盖率 / 指标数字	判卷记录.md 贴原始输出（不许转述）
L2 导师审稿	我	3-5 条：必考项 / 要记住 / 预告项（含“哪里可能自欺”）	判卷记录.md
L3 费曼收口	你讲 + 我追问	三条硬标准（讲清 / 指代码 / 改坏修好）	判卷记录.md + 知识卡

L1 不通过 → 不进 L2/L3；L3 不通过 → 课题不标 ☒，写进 HANDOFF 的“下一句”。

8.2 二、通用红线（继承总纲 + 本学科加严）

1. 结果对 ≠ 满分：本学科分数按“证据链”给，逻辑链完整才满分（总纲 §5a 第 5 条）。
2. 没有预测的实验不算实验：跑之前必须先写“预期现象”（见实验卡），否则 L1 直接判不合格。
3. 无证据的结论不采纳：曲线要原始日志/repro 命令；截图要能追溯到 run.yaml。
4. no-go 不擦除：失败的实验、被推翻的假设全部留在 证据/，并在报告里如实呈现。
5. 口径统一：跨模型比较必须同数据集、同 token 化、同评测脚本；指标优先用词表无关的 bpb。
6. 不许自创指标：新增指标必须给出定义 + 为什么已有的不管用 + 与已有指标的相关性。
7. 误差棒是必填项：任何“变好了”的结论都要给多次运行的范围（至少 2 个种子）。

8. 不许把测试集当验证集反复调（M7 常见自欺）。
9. 掌握的定义（总纲 §5a 第 8 条原文）：当次对 + 隔 1-3 天无提示再对 + 能讲清为什么。
10. 宣布”通过”必须附证据引用：哪份判分输出、哪个数字、哪些 leech 已清零。

8.3 三、每课题的”通过证据”清单（缺一不可）

#	课题	L1 自动判分	L2 审稿关注	L3 收口动作	额外证据
01	BPE 分词器	test_tokenizer.py + test_train_bpe.py 全绿	边界：空串、单字节、中文、emoji、往返	手写一段中文的编码过程并解释	压缩率表（中/英）+ 往返一致性
02	Transformer 骨架	test_model.py、test_nn_util.py 全绿	形状/初始化/数值稳定性	白板上画出一向前向的数据流	三组改坏实验（去 mask / 换位置编码 / 去残差）曲线
03	Scaling Law	自建：拟合优度 + 外推误差	数据点是否同口径、是否外推过远	复述”为什么用幂律而不是线性”	5 点 sweep 原始数据 + 预测先写死再跑的记录
04	0.1B 端到端	自建：loss/bpb 达标 + 采样可读性	是否过拟合/是否记住了测试集	讲清”这个模型会什么、不会什么”	采样样本 + 实测成本 vs 估算偏差
05	Triton kernel	CS336 A2 正确性对齐（误差 < 容差）	数值稳定性、边界 block、dtype	讲清 online softmax 为什么必要	speedup/显存/profile 三图 + 硬件口径声明

#	课题	L1 自动判分	L2 审稿关注	L3 收口动作	额外证据
06	多卡与分布式	扩展效率数据 + 梯度一致性检查	通信占比是否解释得通	讲清“为什么不是 2 倍”	扩展曲线 + 通信时间分解
07	数据清洗去重	自建：规则单测 + 对照组	规则是否过严（伤数据）或过松	讲清每条规则的代价	规则清单 + 有/无清洗的对照
08	SFT/DPO/R1ASL grp. py 全绿 + 三方对照	判定逻辑单测 + rollout 脚本	是否 reward hacking、是否能力回退	逐项解释 GRPO 损失每一项	三方对照表（同口径）+ 失败样本分析
09	可验证环境	判定逻辑单测 + rollout 脚本	假阳性/假阴性样例	讲清“什么算完成”的判定	成功率 + 失败轨迹分类
10	长程 Agent	消融数据 + 轨迹日志	self-judge 是否在放大错误	讲清“什么时候 self-judge 有害”	有无 self-judge 的成功率差
11	结项	报告格式审查 (NeurIPS) demo 可运行	结论是否超出证据范围	自我攻击 3 条 + 现场答疑	英文报告 + demo + 总复盘

8.4 四、成绩口径（对齐唐杰作业的 40% / 60%）

项	权重	内容	判据
平时作业	40%	课题01-09 的 L1+L2+L3	每课题等权，未过者按比例扣

项	权重	内容	判据
大项目	60%	课题04+08+10+11构成的整机（端到端模型 + 对照实验 + Agent 尾巴 + 报告）	问题、动机、方法、结果一项都不能糊（唐杰原话）

注意：课程轨（CORE100299）的成绩由老师给，本表只用来自我标定；但课程大论文（70%）直接取用实训轨 **W8** 前的成果，所以两份成绩是同一份劳动的两个出口。

8.5 五、自欺自查清单（结项前必过）

1. 我报的每个数字，能在 证据/ 里找到原始日志吗？
2. 我的“变好了”，有没有可能是评测口径变了 / 提示词变了 / 只挑了好种子？
3. 我有没有把训练集上的表现当成了能力？
4. 我写下的每个“因为...所以...”，有没有反事实检验（去掉这个因素会怎样）？
5. 我的图，坐标轴和误差棒都标了吗？有没有截断坐标轴让差距显得很大？
6. 我的结论，最强的反方论点是什么？写进报告了吗？

9 上游锁定清单（本学科只读依赖）

立单 2026-09-17。纪律：只记 **commit** + 用途 + 只引哪些路径，不整仓复制（仓库有体积上限）。换机器恢复的方式写在”复现命令”里。引入日期一律记当日，半年后重审一次。

9.1 一、骨架依赖（必须锁，判分与训练都靠它）

仓库	commit	引入日	许可	用途	只用哪些路径
stanford— cs336/ assignment1 —basics	a158843b2010	2026-09-17	MIT	课题01/02 判分器 + handout	tests/ 、cs336_basics /（仅接口 ）、PDF
stanford— cs336/ assignment2 —systems	ca8bc81a59b7	2026-09-17	MIT	课题05/06 判分器	tests/ 、cs336_systems /（空壳） 、PDF
stanford— cs336/ assignment3 —scaling	03 e9372992e9	2026-09-17	MIT	课题03 规 格参考 （API 不 可用，只 读设计）	PDF、cs336_scaling /
stanford— cs336/ assignment4 —data	0555 bea66369	2026-09-17	MIT	课题07 过 滤/去重规 格	tests/ 、configs/ 、PDF
stanford— cs336/ assignment5 — alignment	c2734a263087	2026-09-17	△ 未声明 （用前确 认）	课题08 判 分器 （tests/ test_grpo. py）	tests/ 、data/ 、PDF
stanford— cs336/ lectures	de53a9f979a6	2026-09-17	△ 未声明	讲义 （lecture_01 .py ... lecture_17 .py 、PDF）	全仓可读 （文本量 小）

仓库	commit	引入日	许可	用途	只用哪些 路径
karpthy/ nanochat	92 d63d4e8bb4	2026-09-17	MIT	课 题04/05/06 的工程底 座与对照 实现	nanochat/ 、scripts/ 、runs/ 、tasks/ 、tests/

复现命令（换机器照抄）：

```
mkdir -p 上游 && cd 上游
for r in stanford-cs336/assignment1-basics stanford-cs336/assignment2-
systems \
stanford-cs336/assignment5-alignment karpthy/nanochat; do
n=$(basename $r); git clone --depth 1 https://github.com/$r $n
done
cd assignment1-basics && git fetch --depth 1 origin a158843b2010 && git
checkout a158843b2010
```

9.2 二、知识线依赖（只读，不入仓）

仓库 / 站 点	commit / 版本	引入日	许可	对标模块	用法
rasbt/ reasoning —from— scratch	b2e0841ee8be	2026-09-17	Apache- 2.0	M6/M7/M8	教学法范 本（ch3 先建评测 ）；ch2-4 可 CPU 跑
rasbt/LLMs —from— scratch	ace7c08802b52	2026-09-17	见仓库 (NOASSERTION)	M2-M4	代码级最 细的对照 读物（附 录 GQA/MoE/KV cache）

仓库 / 站点	commit / 版本	引入日	许可	对标模块	用法
huggingface /agents-course	b3946b1d09d2	2026-09-17	Apache-2.0	M8	Agent 三框架 + 最终项目排行榜
harbor-framework /terminal-bench	7 a337a834bff	2026-09-17	Apache-2.0	M8	长程 Agent 基准 (v2; v1 已冻结为 terminal-bench-1)
PrimeIntellect-ai/verifiers	ff3b8dc551ba	2026-09-17	MIT	M7/M8	可验证环境抽象 (课题09 参照, 不引入代码)
jingyaogong /minimind	7 a9137d2e902	2026-09-17	Apache-2.0	M2-M6	中文术语与 RL 脚本对照 (读, 不抄进手写区)
walkinglabs /hands-on-modern-rl	9 f6e429c6894	2026-09-17	△ NOASSERTION	M6	中文 RL→RLVR 教材 (含免费在线 Notebook)
McGill-NLP/nano-aha-moment	5314 e6f8fc60	2026-09-17	MIT	M6	单文件 GRPO (阅读; 已停更, 不作依赖)

仓库 / 站点	commit / 版本	引入日	许可	对标模块	用法
THUDM/slimer	4 c193f1f3750	2026-09-17	Apache-2.0	M8/M9	智谱自家 Agentic RL 基建（W14 后的进阶读物）
KellerJordan/modded-nanogpt	ecbb586296d3	2026-09-17	MIT	M4	单卡 speedrun 思路；注意其验证指标口径争议
datajuicer/data-juicer	1 e9720d01610	2026-09-17	Apache-2.0	M4/M7	课题07 数据清洗的规则参考
d2l-ai/d2l-zh	e6b18ccea714 (2023-08-18)	2026-09-17	Apache-2.0	M0/M1/M3/M5	中文地基读物； $\triangle$ 仓库 2023 后基本停更，内容仍有效但别等更新

9.3 三、许可与合规注意

1. stanford-cs336/assignment5-alignment 与 lectures 仓库未声明 **SPDX** 许可：只做本地学习与对照，不复制进本仓、不再分发。
2. walkinglabs/hands-on-modern-rl 为 NOASSERTION：同样只读、不复制。
3. 所有上游代码一律不进本仓 **git**（体积 + 许可双理由）；本仓只保留 commit 记录与引用的路径清单。
4. 数据：只用 TinyStories（公开）、stanford-cs336/owt-sample（课程公开样本）、以及自己清洗的小语料； $\triangle$ 不要引用 HuggingFaceFW/fineweb / huggingface/fineweb（2026-09-17 实测均 404）。

9.4 四、重审机制

- 每学期期初（下一次：2027-02）重跑一遍 gh api 检查：上游是否改名/归档/换许可；
- CS336 每年改 handout（A5 已从 Qwen-2.5-Math 换为 Olmo-2-1B，新增 GSPO/MaxRL），重审时必须重新读 **CHANGELOG**，否则会拿着旧讲义做新作业。